EXPERIMENT 5

Implement Huffman Coding Algorithm for Lossless Data Compression

Name	HIMANSHU RAJ
Batch	B1 CCVT
SAP ID	590026147
Submitted To	Arjun Abhishek

B.Tech CSE – Cloud Computing & Virtualization (CCVT)

1. Aim

To implement the Huffman Coding algorithm using the greedy approach to generate variable-length binary codes for characters based on their frequencies and demonstrate lossless data compression.

2. Objectives

• Understand the greedy approach used in Huffman Coding.

• Construct a Huffman tree using character frequencies.

• Generate unique, prefix-free binary codes for each character.

• Understand why more frequent characters generally receive shorter codes.

• Analyze the time and space complexity of the implementation.

3. Theory

Huffman Coding is a lossless data compression technique that assigns variable-length binary codes to characters according to their frequencies. More frequent characters generally receive shorter codes, while less frequent characters receive longer codes.

The algorithm repeatedly selects the two uncombined nodes with the smallest frequencies and combines them into a new internal node. This process continues until a single root node remains. During tree traversal, left edges are assigned 0 and right edges are assigned 1.

Huffman codes are prefix-free: no character’s code is the prefix of another character’s code. This property makes the encoded data uniquely decodable.

4. Algorithm

1. Read the number of unique characters and their positive frequencies.

2. Create one leaf node for each character and initialize its left, right, and parent links.

3. Find the two uncombined nodes with the smallest frequencies.

4. Create a new internal node with frequency equal to the sum of the two selected frequencies.

5. Make the selected nodes the left and right children of the new node and mark them as combined.

6. Repeat until only one root node remains.

7. Traverse the Huffman tree, assigning 0 to each left edge and 1 to each right edge.

8. Display the Huffman code for each character.

5. Sample Input and Output

Sample input: 6 characters with the following frequencies.

Character	Frequency
A	5
B	9
C	12
D	13
E	16
F	45

Generated Huffman codes:

Character	Frequency	Huffman Code
A	5	1100
B	9	1101
C	12	100
D	13	101
E	16	111
F	45	0

The program completed successfully and displayed the generated Huffman codes for all six characters.

6. Complexity Analysis

Operation	Time Complexity
Finding two minimum-frequency nodes	O(n) per iteration
Building the Huffman tree	$O(n^2)$
Generating codes by tree traversal	O(n)
Overall time complexity	$O(n^2)$
Space complexity	O(n)

7. Performance Discussion

This implementation finds the two minimum-frequency nodes using a linear scan during each iteration. Since there are n − 1 combinations and each scan takes O(n) time, tree construction takes $O(n^2)$ time. A min-heap implementation can reduce tree construction time to O(n log n).

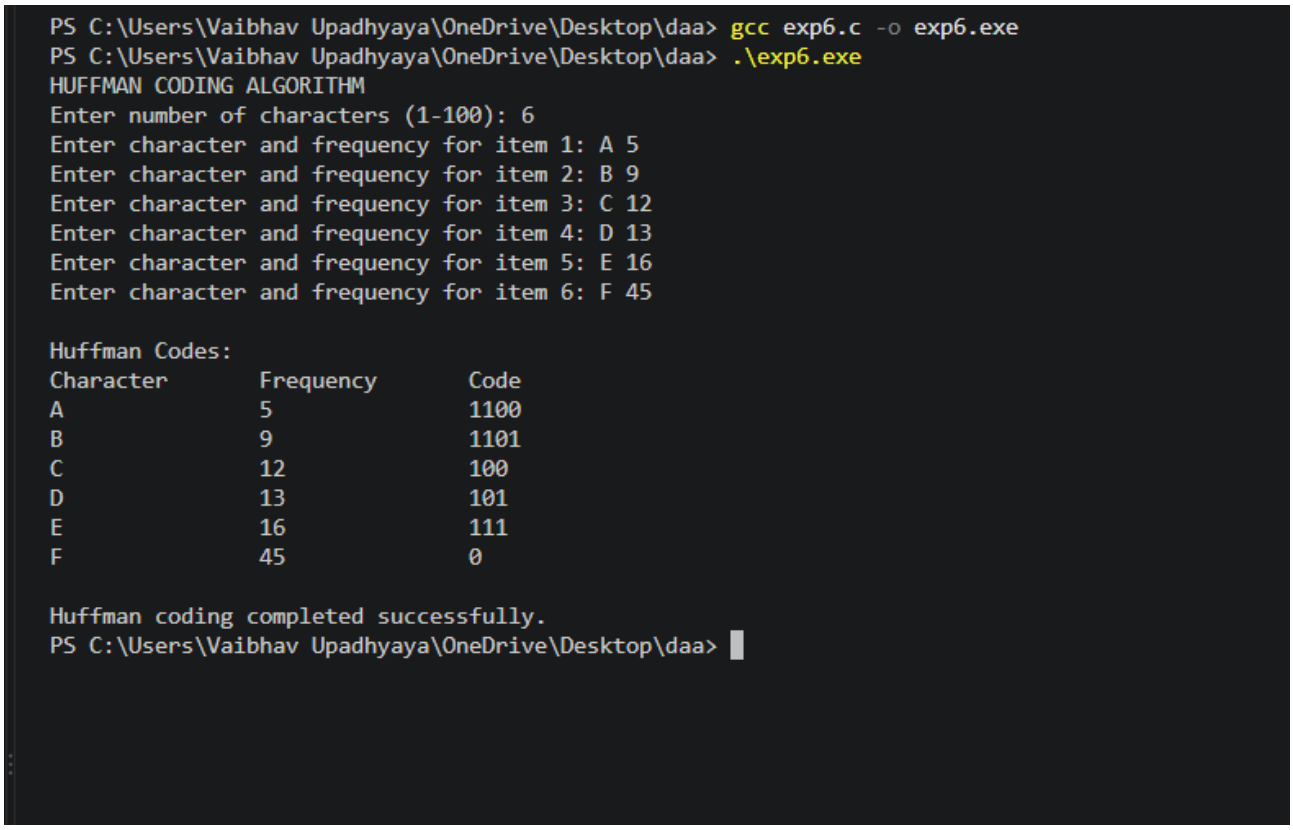
8. Result

Huffman Coding was implemented successfully in C using the greedy approach. For the sample frequencies A=5, B=9, C=12, D=13, E=16, and F=45, the program generated prefix-free variable-length binary codes for all characters.

9. Screenshots / Evidence

The following screenshots document the Huffman Coding implementation and successful execution in Visual Studio Code. Source code is omitted from this report, following the format of the previous experiment reports.

Screenshot 1



Screenshot 1: Node structure, data arrays, and function to find the two minimum-frequency nodes.

Screenshot 2

exp6.c > main(void)

```c

#include <stdio.h>

#define MAX 100
#define TOTAL (2 * MAX - 1)

struct Node {
    char ch;
    int freq;
    int left;
    int right;
    int parent;
};

struct Node tree[TOTAL];
char codes[MAX][MAX];
int n;

// Find the two minimum-frequency uncombined nodes
Windsurf: Refactor | Explain | ×
void findMin(int limit, int *first, int *second) {
    *first = -1;
    *second = -1;

    for (int i = 0; i < limit; i++) {
        if (tree[i].parent != -1)
            continue;

        if (*first == -1 ||
            tree[i].freq < tree[*first].freq) {
            *second = *first;
            *first = i;
        } else if (*second == -1 ||
                   tree[i].freq < tree[*second].freq) {
            *second = i;
        }
    }
}

// Construct the Huffman tree
Windsurf: Refactor | Explain | ×
void buildHuffmanTree(void) {
    int totalNodes = n;

    while (totalNodes < 2 * n - 1) {
        int first, second;
        findMin(totalNodes, &first, &second);

```

Screenshot 2: Construction of the Huffman tree and creation of combined frequency nodes.

Screenshot 3



```
        tree[totalNodes].ch = '\0';
        tree[totalNodes].freq =
            tree[first].freq + tree[second].freq;
        tree[totalNodes].left = first;
        tree[totalNodes].right = second;
        tree[totalNodes].parent = -1;

        tree[first].parent = totalNodes;
        tree[second].parent = totalNodes;

        totalNodes++;
    }
}

// Generate codes by traversing the tree
Windsurf: Refactor | Explain | ×
void generateCodes(int node, char code[], int depth) {
    if (tree[node].left == -1 &&
        tree[node].right == -1) {

        // Handle the case of a single character
        if (depth == 0) {
            code[depth++] = '0';
        }

        code[depth] = '\0';

        for (int i = 0; i < n; i++) {
            if (tree[i].ch == tree[node].ch) {
                int j = 0;
                while (code[j] != '\0') {
                    codes[i][j] = code[j];
                    j++;
                }
                codes[i][j] = '\0';
                break;
            }
        }
        return;
    }

    code[depth] = '0';
    generateCodes(tree[node].left, code, depth + 1);

    code[depth] = '1';
```

Screenshot 3: Recursive traversal used to generate prefix-free binary codes.

Screenshot 4

```
    generateCodes(tree[node].right, code, depth + 1);
}

Windsurf: Refactor | Explain | Generate Function Comment | ×
int main(void) {
    char code[MAX];
    int totalNodes;

    printf("HUFFMAN CODING ALGORITHM\n");
    printf("Enter number of characters (1-%d): ", MAX);

    if (scanf("%d", &n) != 1 || n < 1 || n > MAX) {
        printf("Invalid number of characters!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        printf("Enter character and frequency for item %d: ",
               i + 1);

        if (scanf(" %c %d", &tree[i].ch,
                  &tree[i].freq) != 2 ||
            tree[i].freq <= 0) {
            printf("Invalid character or frequency!\n");
            return 1;
        }

        for (int j = 0; j < i; j++) {
            if (tree[j].ch == tree[i].ch) {
                printf("Characters must be unique!\n");
                return 1;
            }
        }

        tree[i].left = -1;
        tree[i].right = -1;
        tree[i].parent = -1;
        codes[i][0] = '\0';
    }

    buildHuffmanTree();

    totalNodes = 2 * n - 1;
    int root = totalNodes - 1;

    generateCodes(root, code, 0);
```

Screenshot 4: Main function, input validation, tree construction, and code-generation call.

Screenshot 5

```
136
137     printf("\nHuffman Codes:\n");
138     printf("Character\tFrequency\tCode\n");
139
140     for (int i = 0; i < n; i++) {
141         printf("%c\t\t%d\t\t%s\n",
142                 tree[i].ch, tree[i].freq, codes[i]);
143     }
144
145     printf("\nHuffman coding completed successfully.\n");
146
147     return 0;
148 }
149
```

Screenshot 5: Successful terminal output showing the generated Huffman codes for the sample frequencies.